

Industrial AI Labs

Guía de Desarrollo IA y Estructura de Agentes

Documento ID: IAL-DEV-IA-2026

Versión: 2.1

Para: Desarrolladores de Software e Ingenieros de Datos

Clasificación: Confidencial Interno

1. Entorno de Ejecución Python

El estándar oficial de programación para todos los microservicios de inteligencia artificial en la compañía se basa en Python. Mantenemos una estricta homogeneidad de versiones en todos los entornos de desarrollo y producción para mitigar inconsistencias de dependencias.

La versión mandatoria actual es **Python 3.11.x**. No se permite el uso de versiones inferiores a la 3.11 debido a requerimientos nativos de optimización de memoria de los SDKs de LLM.

El gestor de dependencias oficial es Poetry. Todo proyecto debe incluir un archivo `pyproject.toml` correctamente estructurado.

2. Arquitectura de Orquestación con LangChain

Para la construcción de pipelines de datos orientados a lenguaje natural y agentes autónomos, se utiliza el framework **LangChain**. Estructuramos las interacciones usando LCEL (LangChain Expression Language) para facilitar el streaming de tokens y la modularidad.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_google_genai import ChatGoogleGenerativeAI

prompt = ChatPromptTemplate.from_template("Analiza el estado del sensor industrial:
{sensor_data}")
model = ChatGoogleGenerativeAI(model="gemini-1.5-pro")
chain = prompt | model | StrOutputParser()
```

3. Integración con la API de Gemini (Google)

Industrial AI Labs utiliza los modelos fundacionales **Gemini 1.5 Pro** y **Gemini 1.5 Flash** para el procesamiento multimodal de telemetría y reportes industriales.

Al instanciar el cliente, asegúrese de ajustar los parámetros de configuración para garantizar la reproducibilidad y el control de costos corporativos:

- `temperature`: Fijar en 0.0 para análisis de datos y reportes de fallas técnicas; máximo 0.5 para asistencia general.
- `max_output_tokens`: Controlar según el caso de uso para optimizar la latencia en transmisiones en vivo.

4. Contenedores con Docker y Aislamiento de Entornos

Ningún código de IA se ejecuta directamente en el sistema anfitrión de producción. Todo desarrollo debe empaquetarse utilizando Docker aplicando el principio de imágenes multi-etapa (multi-stage builds) para reducir la superficie de ataque y el tamaño físico del artefacto final.

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY poetry.lock pyproject.toml ./
RUN pip install poetry && poetry install --no-root

FROM python:3.11-slim AS runner
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/
python3.11/site-packages
COPY . .
CMD ["python", "main.py"]
```

5. Flujo de Control de Versiones con Git

Implementamos una estrategia estricta de GitFlow modificada. Está terminantemente prohibido empujar cambios de manera directa a la rama `main` o `develop`. Todo cambio debe originarse desde un feature branch y requiere la aprobación técnica obligatoria de al menos dos ingenieros Senior mediante un Pull Request (PR).

6. Buenas Prácticas de Código y Estándares de Diseño

Todo script debe cumplir con las siguientes directrices de calidad:

1. **Tipado Estático:** Uso exhaustivo de hints de tipos (Type Hinting) nativos de Python.
2. **Linter y Formateador:** Ejecución obligatoria de `ruff` o `black` antes de realizar un commit.
3. **Manejo de Excepciones:** No se permiten bloques `try/except` `Exception` vacíos. Cada excepción de red de la API de Gemini debe registrarse con su respectivo stack trace en la herramienta de monitoreo central.